

เอกสารโครงสร้างฐานข้อมูล

ระบบจัดเส้นทางและแบ่งงานไรเดอร์อัจฉริยะ

WebAPI Assignment 5

ER Diagram รุ่น V3 และ Data Dictionary

จัดทำเพื่อใช้เป็นต้นแบบก่อนพัฒนา Backend และฐานข้อมูลจริง

เอกสารฉบับนี้อธิบายตาราง ฟิลด์ ความสัมพันธ์ กฎธุรกิจ สูตรต้นทุน
และข้อกำหนดข้อมูลตัวอย่างที่จำเป็นต่อระบบตาม Project.pdf

ฉบับ V3 | 26 กันยายน 2026

1. ภาพรวมและขอบเขต

โครงสร้างนี้รองรับระบบจัดการลูกค้า ออเดอร์ ไรเดอร์ และการคำนวณเส้นทางหลายเวอร์ชัน

รายการ	รายละเอียด
ไฟล์ ERD	WebAPI_Assignment5_PROJECT_FINAL_V3.erdplus
จำนวน Entity	6 ตาราง: STORES, CUSTOMERS, ORDERS, RIDERS, ROUTE_PLANS, ROUTE_STOPS
จำนวนความสัมพันธ์	5 ความสัมพันธ์แบบ 1:N
ฐานข้อมูลเป้าหมาย	MySQL หรือฐานข้อมูลเชิงสัมพันธ์ที่รองรับ FK และ CHECK constraint
ขอบเขต	จัดการข้อมูลลูกค้า ออเดอร์จำลอง การแบ่งจุดส่งให้ไรเดอร์ และประเมินต้นทุน/กำไร

การเปลี่ยนแปลงจาก ER เดิม

รายการ	รายละเอียด
เพิ่มตาราง STORES	เก็บชื่อร้าน เบอร์โทร ที่อยู่ และพิกัดร้าน เพื่อรองรับการแก้ไขข้อมูลร้าน
เพิ่ม ROUTE_PLANS.store_id	FK ไปยัง STORES ทำให้ทุกแผนเส้นทางมีต้นทางเป็นร้านที่ระบุได้
ส่วนที่คงเดิม	CUSTOMERS, ORDERS, RIDERS, ROUTE_STOPS และกติกาไรเดอร์ไม่เกิน 3 ออเดอร์

หมายเหตุ: ROUTE_STOPS.rider_id ถูกเก็บไว้เพื่อกำหนดว่าแต่ละจุดส่งเป็นของไรเดอร์คนใด ทำให้ระบบวาดเส้นทางแยกสีและกรองใบงานของไรเดอร์ได้ ส่วนกติกาไม่เกิน 3 ออเดอร์ต่อไรเดอร์ต้องตรวจสอบซ้ำใน API ก่อนบันทึกแผน

2. ความสัมพันธ์ของ ER Diagram

แผนภาพใน ERDPlus ใช้ความสัมพันธ์ 5 รายการด้านล่าง โดย Optional Many หมายถึงฝั่งนั้นอาจยังไม่มีข้อมูลหรือมีได้หลายรายการ และ Mandatory One หมายถึงข้อมูลปลายทางต้องอ้างอิงหนึ่งรายการเสมอ

ความสัมพันธ์	คาร์ดินาลิตี	ความหมายในการทำงาน
CUSTOMERS -> ORDERS	0..N -> 1	ลูกค้าหนึ่งคนมีหลายออเดอร์ได้ แต่ออเดอร์ต้องมีลูกค้า 1 คน
STORES -> ROUTE_PLANS	0..N -> 1	ร้านหนึ่งมีหลายแผนได้ แต่แผนหนึ่งใช้ร้านต้นทางเดียว
ROUTE_PLANS -> ROUTE_STOPS	0..N -> 1	แผนหนึ่งมีจุดส่งหลายจุด และจุดส่งอยู่ในแผนเดียว
RIDERS -> ROUTE_STOPS	0..N -> 1	จุดส่งแต่ละจุดถูกมอบหมายให้ไรเดอร์หนึ่งคน
ORDERS -> ROUTE_STOPS	0..N -> 1	ออเดอร์อาจอยู่ในหลายเวอร์ชันของเส้นทางได้ แต่จุดส่งหนึ่งจุดอ้างอิงออเดอร์เดียว

ลำดับการทำงานของข้อมูล

ลำดับ	ข้อมูล	บทบาท
1	STORES	ร้านเป็นจุดเริ่มต้นของแผนเส้นทาง
2	ROUTE_PLANS	ระบบสร้างแผนหลาย version เพื่อเปรียบเทียบเส้นทาง
3	ROUTE_STOPS	แผนถูกแบ่งเป็นจุดส่งและลำดับการเดินทาง
4	RIDERS	แต่ละจุดส่งถูกมอบหมายให้ไรเดอร์ โดยรวมไม่เกิน 3 ออเดอร์ต่อคน
5	ORDERS -> CUSTOMERS	แต่ละจุดส่งอ้างอิงออเดอร์และข้อมูลพิกัดลูกค้า

3.1 ตาราง STORES

Data dictionary สำหรับใช้เขียน SQL, Model และ API

ฟิลด์	ชนิดข้อมูล	คีย์/ข้อกำหนด	คำอธิบาย
store_id	INT	PK, AUTO_INCREMENT	รหัสร้าน
store_name	VARCHAR(150)	NOT NULL	ชื่อร้าน
phone	VARCHAR(20)	NOT NULL	เบอร์โทรร้าน
address	VARCHAR(255)	NOT NULL	ที่อยู่ร้าน
latitude	DECIMAL(10,7)	NOT NULL	ละติจูดของร้าน
longitude	DECIMAL(10,7)	NOT NULL	ลองจิจูดของร้าน
is_active	BOOLEAN	DEFAULT TRUE	สถานะร้านที่ใช้งานอยู่
created_at	DATETIME	NOT NULL	วันที่สร้างข้อมูล
updated_at	DATETIME	NOT NULL	วันที่แก้ไขล่าสุด

3.2 ตาราง CUSTOMERS

Data dictionary สำหรับใช้เขียน SQL, Model และ API

ฟิลด์	ชนิดข้อมูล	คีย์/ข้อกำหนด	คำอธิบาย
customer_id	INT	PK, AUTO_INCREMENT	รหัสลูกค้า
first_name	VARCHAR(100)	NOT NULL	ชื่อ
last_name	VARCHAR(100)	NOT NULL	นามสกุล
phone	VARCHAR(20)	NOT NULL	เบอร์โทรศัพท์
address	VARCHAR(255)	NOT NULL	ที่อยู่หรือจุดรับอาหาร
latitude	DECIMAL(10,7)	NOT NULL	ละติจูดลูกค้า
longitude	DECIMAL(10,7)	NOT NULL	ลองจิจูดลูกค้า
created_at	DATETIME	NOT NULL	วันที่เวลาที่สร้างข้อมูล
updated_at	DATETIME	NOT NULL	วันที่เวลาที่แก้ไขล่าสุด

3.3 ตาราง ORDERS

Data dictionary สำหรับใช้เขียน SQL, Model และ API

ฟิลด์	ชนิดข้อมูล	คีย์/ข้อกำหนด	คำอธิบาย
order_id	INT	PK, AUTO_INCREMENT	รหัสออเดอร์
customer_id	INT	FK -> CUSTOMERS, NOT NULL	ลูกค้าที่สั่งอาหาร
box_quantity	TINYINT UNSIGNED	CHECK 1-3, NOT NULL	จำนวนกล่องในออเดอร์
order_date	DATE	NOT NULL	วันที่ของออเดอร์
status	VARCHAR(20)	NOT NULL	PENDING, ASSIGNED, DELIVERING, DELIVERED, CANCELLED
created_at	DATETIME	NOT NULL	เวลาที่สร้างออเดอร์

3.4 ตาราง RIDERS

Data dictionary สำหรับใช้เขียน SQL, Model และ API

ฟิลด์	ชนิดข้อมูล	คีย์/ข้อกำหนด	คำอธิบาย
rider_id	INT	PK, AUTO_INCREMENT	รหัสไรเดอร์
name	VARCHAR(150)	NOT NULL	ชื่อไรเดอร์
phone	VARCHAR(20)	NOT NULL	เบอร์โทรไรเดอร์
vehicle_plate	VARCHAR(20)	NOT NULL	ทะเบียนรถ
status	VARCHAR(20)	NOT NULL	AVAILABLE, BUSY, INACTIVE
max_orders_per_route	TINYINT UNSIGNED	DEFAULT 3, CHECK <= 3	จำนวนออเดอร์สูงสุดต่อรอบ

3.5 ตาราง ROUTE_PLANS

Data dictionary สำหรับใช้เขียน SQL, Model และ API

ฟิลด์	ชนิดข้อมูล	คีย์/ข้อกำหนด	คำอธิบาย
route_plan_id	INT	PK, AUTO_INCREMENT	รหัสแผนเส้นทาง
store_id	INT	FK -> STORES, NOT NULL	ร้านต้นทางของแผน
route_date	DATE	NOT NULL	วันที่จัดส่ง
version	INT	NOT NULL, >= 1	เวอร์ชันการคำนวณเส้นทาง
status	VARCHAR(20)	NOT NULL	DRAFT, SELECTED, REJECTED, COMPLETED
total_distance	DECIMAL(10,2)	>= 0	ระยะทางรวมกิโลเมตร
estimated_minutes	INT	>= 0	เวลาส่งโดยประมาณ
total_cost	DECIMAL(10,2)	>= 0	ต้นทุนรวมของแผน
total_revenue	DECIMAL(10,2)	>= 0	รายรับรวมของแผน
total_profit	DECIMAL(10,2)	คำนวณได้	กำไรหรือขาดทุนของแผน
calculated_at	DATETIME	NOT NULL	เวลาที่คำนวณแผน
worksheet_code	VARCHAR(30)	UNIQUE, NOT NULL	เลขใบงานสำหรับไรเดอร์

ข้อสังเกต: store_id เป็นการเปลี่ยนแปลงใหม่จาก ER เดิมและเป็น FK สำคัญ เพราะแผนเส้นทางต้องคำนวณจากพิกัดร้านที่แก้ไขได้

3.6 ตาราง ROUTE_STOPS

Data dictionary สำหรับใช้เขียน SQL, Model และ API

ฟิลด์	ชนิดข้อมูล	คีย์/ข้อกำหนด	คำอธิบาย
route_stop_id	INT	PK, AUTO_INCREMENT	รหัสจุดส่ง
route_plan_id	INT	FK -> ROUTE_PLANS, NOT NULL	แผนเส้นทางที่จุดส่งอยู่
rider_id	INT	FK -> RIDERS, NOT NULL	ไรเดอร์ที่รับจุดส่งนี้
order_id	INT	FK -> ORDERS, NOT NULL	ออเดอร์ที่ต้องนำส่ง
stop_sequence	INT	NOT NULL, >= 1	ลำดับจุดส่ง
distance_from_previous	DECIMAL(10,2)	>= 0	ระยะจากจุดก่อนหน้าเป็นกิโลเมตร
estimated_arrival	DATETIME	NULL ได้	เวลาถึงโดยประมาณ
delivery_status	VARCHAR(20)	NOT NULL	PENDING, DELIVERED, FAILED
delivered_at	DATETIME	NULL ได้	เวลาส่งสำเร็จ

ข้อสังเกต: ออเดอร์เดียวกันอาจปรากฏใน ROUTE_STOPS หลายครั้งเมื่อมีการคำนวณเส้นทางหลาย version ดังนั้น API ต้องระบุว่า version ใดเป็น SELECTED ก่อนนำไปแสดงผลจริง

4. กฎธุรกิจและสูตรคำนวณ

หัวข้อ	กติกาที่ระบบต้องบังคับใช้
จำนวนออเดอร์จำลอง	วันละ 20-30 ออเดอร์
จำนวนกล่องต่อออเดอร์	1-3 กล่องต่อออเดอร์
จำนวนออเดอร์ต่อไรเดอร์	ไม่เกิน 3 ออเดอร์ต่อแผนเส้นทาง
พื้นที่ลูกค้า	ลูกค้าอยู่ในรัศมีประมาณ 3 กิโลเมตรจากร้าน
ค้นหาลูกค้า	ค้นหาลูกค้าในรัศมี 1 กิโลเมตรจากพิกัดที่กำหนด
ค้นหาออเดอร์	แสดงออเดอร์ในรัศมี 2 กิโลเมตรจากพิกัดที่กำหนด
เวลาให้บริการ	เริ่มจัดส่งประมาณ 11:30 น. และต้องถึงภายใน 12:30 น.
การคำนวณซ้ำ	สร้าง route plan หลาย version ได้จนกว่าเจ้าของร้านจะเลือกแผนที่พอใจ

สูตรต้นทุน รายรับ และกำไร

รายการ	สูตรคำนวณ
รายรับ	จำนวนกล่องรวม x 65 บาท
ต้นทุนอาหาร	จำนวนกล่องรวม x 40 บาท
ค่ารอบไรเดอร์	15 บาทต่อไรเดอร์ต่อรอบ
ค่าส่งตามระยะทาง	ระยะทางรวม x จำนวนกล่องที่ไรเดอร์รับ x 2 บาท
ต้นทุนรวม	ต้นทุนอาหาร + ค่ารอบไรเดอร์ + ค่าส่งตามระยะทาง
กำไร/ขาดทุน	รายรับ - ต้นทุนรวม

ค่า total_cost, total_revenue และ total_profit ใน ROUTE_PLANS ควรเป็น snapshot ของผลคำนวณใน version นั้น เพื่อให้ย้อนดูและเปรียบเทียบแผนที่เคยคำนวณได้

5. ข้อมูลตัวอย่างและการตรวจสอบความถูกต้อง

แนวทาง Seed Data

ชุดข้อมูล	รายละเอียดที่ควรเตรียม
ร้าน	อย่างน้อย 1 ร้าน พร้อมพิกัดตั้งต้นที่แก้ไขได้
ลูกค้า	ชื่อสมมติ เบอร์สมมติ ที่อยู่สมมติ และพิกัดแบบกลุ่มใกล้เคียงกันรอบพื้นที่มหาวิทยาลัยมหาสารคาม
ออเดอร์	สร้าง 20-30 รายการต่อวัน โดย box_quantity อยู่ระหว่าง 1-3
ไรเดอร์	เตรียมหลายคนเพื่อทดสอบการแบ่งกลุ่ม และกำหนด max_orders_per_route = 3
เส้นทาง	เก็บหลาย version เพื่อทดสอบป้อนคำนวณเส้นทางใหม่

ดัชนีและข้อกำหนดที่แนะนำ

ตาราง/ฟิลด์	ดัชนีหรือข้อกำหนดที่แนะนำ
CUSTOMERS	INDEX(first_name, last_name), INDEX(latitude, longitude)
ORDERS	INDEX(customer_id), INDEX(order_date, status), CHECK(box_quantity BETWEEN 1 AND 3)
RIDERS	INDEX(status), CHECK(max_orders_per_route BETWEEN 1 AND 3)
ROUTE_PLANS	UNIQUE(worksheet_code), INDEX(store_id, route_date, version)
ROUTE_STOPS	UNIQUE(route_plan_id, stop_sequence), INDEX(route_plan_id, rider_id), INDEX(order_id)
STORES	INDEX(is_active), CHECK(latitude BETWEEN -90 AND 90), CHECK(longitude BETWEEN -180 AND 180)

เช็กลิสต์ก่อนเริ่มเขียน Backend

- สร้าง STORES ก่อนสร้าง ROUTE_PLANS และกำหนดพิกัดร้านให้ครบ
- เพิ่ม FK และ CHECK constraint ตาม Data Dictionary
- ตรวจสอบระยะทางด้วยสูตร Haversine ก่อนรับลูกค้าหรือออเดอร์เข้าระบบ
- ตรวจสอบจำนวนกล่องต่อออเดอร์ 1-3 และจำนวนออเดอร์ต่อไรเดอร์ไม่เกิน 3
- เมื่อคำนวณเส้นทางใหม่ให้สร้าง version ใหม่ ไม่ทับข้อมูลเดิม
- เลือก route plan เพียง version เดียวเป็น SELECTED สำหรับการจัดส่งจริง
- สร้าง API สำหรับ CRUD ลูกค้า, CRUD ออเดอร์, ล้างออเดอร์จำลอง และดูเส้นทางตามรหัส

สถานะเอกสาร: ERD และ Data Dictionary จัดทำเสร็จแล้ว ส่วน SQL schema, seed data 20-30 ออเดอร์ และ Backend จะทำในขั้นตอนถัดไปโดยยึดโครงสร้างนี้เป็นหลัก